

GIẢI THÍCH CHI TIẾT KIẾN TRÚC MÃ NGUỒN HYBRID FTP SERVER

Tài liệu này tổng hợp các phân tích và giải nghĩa chuyên sâu về từng file mã nguồn trong hệ thống Hybrid FTP Server. Nội dung tập trung làm rõ vai trò của từng thành phần, cấu trúc thiết kế hướng đối tượng, và đặc biệt là các rủi ro bảo mật cùng các bẫy giao thức đang được để lại như một thử thách (TODO) cho quá trình hoàn thiện.

1. Nhóm Cấu hình và Hướng dẫn

- **CMakeLists.txt:** Cấu hình hệ thống build gọn gàng và chuẩn mực cho dự án C++17. Việc tích hợp sẵn thư viện đa luồng (Threads::Threads) và quy hoạch rõ ràng cấu trúc thư mục giúp cho quá trình biên dịch qua terminal trên các môi trường như MacOS trở nên trơn tru, hạn chế tối đa các lỗi liên quan đến đường dẫn file.
- **README_vi.md:** Đóng vai trò là bản thiết kế thi công. File này không chỉ hướng dẫn chạy thử mà còn vạch rõ ranh giới kiến trúc, khoanh vùng chính xác những lỗ hổng "có tình để trống" (như buffer TCP, Directory Traversal, MDTM) để định hướng trọng tâm cho các câu hỏi vấn đáp bảo vệ kỹ thuật.

2. Nhóm Lõi (Core) và Điều phối Luồng

- **main.cpp:** Trái tim của hệ thống. Quản lý vòng lặp accept() và khởi tạo luồng song song thông qua std::thread(...).detach(). Điểm tối quan trọng ở file này là cạm bẫy TCP Buffer: mã nguồn hiện tại đang ép 1 lần recv() thành 1 lệnh hoàn chỉnh. Do TCP truyền dữ liệu dạng luồng (stream), bắt buộc phải có thuật toán cấp phát bộ đệm tinh tế để liên tục thu thập và cắt chuỗi chính xác theo ký tự \r\n.
- **command_dispatcher.h & .cpp:** Hoạt động như một "Single Entry Point" (Điểm tiếp nhận duy nhất). Nó phân tích dòng lệnh thô thành động từ (verb) và tham số (args), sau

đó định tuyến luồng xử lý. Kỹ thuật này giúp phân tách rạch ròi trạng thái lệnh được phép chạy trước và sau khi xác thực.

- **session.h**: Lưu trữ toàn bộ bối cảnh của một client. Điểm sáng trong thiết kế là mỗi thread sở hữu duy nhất một Session, loại bỏ nhu cầu sử dụng mutex bên trong cấu trúc này để tránh deadlock. Sự đồng bộ (Concurrency Control) chỉ cần thiết ở cấp độ danh sách quản lý toàn cục khi thống kê các client đang kết nối.

3. Nhóm Giao tiếp Mạng và Tiện ích

- **rdt_interface.h & mock_rdt (h/cpp)**: Đại diện cho tư duy Dependency Inversion. Lớp IRDTChannel định ra bản hợp đồng giao tiếp thuần ảo, giúp các module phía trên không bị phụ thuộc vào tầng mạng bên dưới. MockRDTChannel tận dụng socket UDP và hàm connect() để cố định địa chỉ đích mà không cần bắt tay (handshake), tạo ra một kênh truyền "giả" không tin cậy để phục vụ kiểm thử song song.
- **reply_codes.h & .cpp**: Chuẩn hóa toàn bộ phản hồi bằng cách gán tên cho các mã FTP 3 chữ số, giúp triệt tiêu hoàn toàn các "magic numbers" (những con số cứng vô nghĩa) trong code, đảm bảo tính dễ đọc và bảo trì.

4. Các Module Xử lý Nghiệp vụ (Handlers)

File	Giải thích & Thách thức Kỹ thuật
fs_handler (h/cpp)	Quản lý toàn bộ metadata và thao tác điều hướng. Trọng tâm bảo mật nằm ở hàm resolveSafePath giúp chặn đứng hành vi Directory Traversal (lợi dụng .. để trở ra ngoài root). Tư duy xử lý logic đường dẫn ở

File	Giải thích & Thách thức Kỹ thuật
	đây hoàn toàn tương đồng với việc kiểm soát đệ quy và liên kết nút trong các cấu trúc cây (như cây nhị phân tìm kiếm), nơi bạn tuyệt đối không được phép duyệt ngược quá giới hạn của nút gốc (root node). Điểm yếu cần vá (TODO): Lệnh MKD hiện chưa đi qua lớp lá chắn bảo vệ này. Đồng thời, hàm MDTM đòi hỏi sự thông thạo trong việc ép kiểu thời gian <code>std::filesystem::file_time_type</code> cực kỳ lắt léo của C++17.
transfer_handler (h/cpp)	Xử lý luồng I/O nhị phân cốt lõi. Các lệnh RETR và STOR chia file thành từng phần nhỏ (chunks) để đẩy qua kênh RDT. Điểm yếu cần vá (TODO): Kích thước <code>CHUNK_SIZE</code> cần được tính toán tinh chỉnh lại để tối ưu với MTU của UDP (khoảng 1400 byte). Hàm hiện tại cũng đang bị hỏng bảo mật đường dẫn và thiếu thao tác gửi mã tín hiệu mở kênh 150 trước khi truyền tải thực sự.

File	Giải thích & Thách thức Kỹ thuật
auth_handler (h/cpp)	Vận hành như một máy trạng thái (state machine) đơn giản để kiểm soát quá trình xác thực. Hiện tại logic handlePASS đang được thiết lập mở hoàn toàn để phục vụ test; trong thực tế, cần được tích hợp với một cấu trúc dữ liệu bản đồ băm (hash map) hoặc đọc file cấu hình để đối chiếu danh tính.